Protecting data content

Even with appropriate security measures in place, a system and its data might still be accessed by someone who overcomes security to break into the system. This type of access can still be made a waste of time and effort if the stored data cannot be read. Data can be encrypted to ensure this. Some details of encryption methods are discussed in Chapter 21 (Sections 21.01, 21.03, 21.04, 21.05 & 21.06).

9.05 Data validation and verification

Data integrity can never be guaranteed, but the chances are improved if appropriate measures are taken when data originally enters a system or when it is transmitted from one system to another.

Validation of data entry

The term **validation** is a somewhat misleading one. It seems to imply that data is accurate if it has been validated. This is far from the truth. For example, if entry of a name is expected but the wrong name is entered, it will still be recognised as a name and therefore accepted as valid. Validation can only prevent incorrect data if there is an attempt to input data that is of the wrong type, in the wrong format or out of range.

Data validation is implemented by software associated with a data entry interface. There are a number of different types of check that can be made. Typical examples are:

- a presence check to ensure that an entry field is not left blank
- a format check, for example a date has to be dd/mm/yyyy
- a length check, for example with a telephone number
- a range check, for example the month in a date must not exceed 12
- a limit check, for example a maximum number of years for a person's age
- a type check, for example only a numeric value for the month in a date
- an existence check, for example that a file exists with the filename referred to in the data entry.

Verification of data entry

When data is entered into a system, **verification** means getting the user to confirm that the data entered was what was intended to be entered. Unfortunately, this still does not mean that the data entered is correct. Double entry is one method of verification. The most common example is when a user is asked to supply a new password. There will usually be a request for the password to be re-entered. A second method is to use a visual check of what has been entered. If a form has been filled in, it always makes sense to read through the data entered before sending the form off to its destination.

Check digit

When a series of numbers are used to identify something, it is possible to use a check digit method of verification. There are many different options here but they all require a calculation to be made with the numbers that have been entered. The final part of the calculation involves an integer division from which the remainder is added as an extra one or two digits at the end of the series of numbers. In one scheme where only a single check digit is allowed, the letter X is used when the remainder is calculated as 10. When the data is subsequently read the same calculation is carried out and the result is compared to the check digit that had been stored. This technique is used for a bar code or for the ISBN for a book.

KEY TERMS

Validation: a check that data entered is of the correct type and format; it does not guarantee that data is accurate

Verification: confirmation of data received by a system

Verification during data transfer

It is possible for data to be corrupted during transmission. Often, this happens when an individual bit is flipped from 1 to 0 or from 0 to 1. Verification techniques need to check on some property associated with the bit pattern.

The simplest approach is to use a simple one-bit parity check. This is particularly easy to do if data is transferred in bytes using a seven-bit code. Either even or odd parity can be implemented in the eighth bit of the byte. Assuming even parity, this is the procedure.

1. At the transmitting end, the number of 1s in the seven-bit code is counted.
2. If the count gives an even number, the parity bit is set to 0.
3. If the count gives an odd number, the parity bit is set to 1.
4. This is repeated for every byte in the transmission.
5. At the receiving end, the number of 1s in the eight-bit code is counted.
6. If the count gives an even number, the byte is accepted.
7. This is repeated for every byte in the transmission.

If no errors are found, the transmission is accepted. However, we cannot guarantee that the transmission is error free. It is possible for two bits to be flipped in an individual byte, which would mean that the transmission is incorrect but the parity check is passed. Fortunately, this is rather unlikely so it is sensible to assume no error. The limitation of the method is that it can only detect the presence of an error. It cannot identify the actual bit that is in error. If an error is detected, re-transmission has to be requested.

An alternative approach is to use the checksum method. At the transmitting end a block is defined as a number of bytes. Then, no matter what the bytes represent, the bits in each byte are interpreted as a binary number. The sum of these binary numbers in a block is calculated and supplied as a checksum value in the transmission. This is repeated for each block. The receiver does the same calculation and checks the sum of the numbers with the checksum value transmitted for each block in turn. Once again, an error can be detected but its position in the transmission cannot be determined.

TIP

Don't confuse a check digit with a checksum. A check digit is used for stored data; a checksum is only used for transmitted data.

Detecting the exact position of an error so as to correct it is considerably more complex. One approach is to use the parity block check method. Like the checksum method this is a longitudinal parity meaning that it is used to check a sequence of binary digits contained in a number of bytes.

WORKED EXAMPLE 9.01

Using a parity block check

At the transmitting end, a program reads a group of seven bytes as illustrated in Figure 9.01. The data is represented by seven bits for each byte. The most significant bit in each byte, bit 7, is undefined so we have left it blank.

	Seven-bit codes						
	1	0	1	0	0	1	1
	0	1	1	0	0	0	1
	1	0	1	1	0	0	0
	0	0	1	1	1	0	0
	0	1	1	0	0	1	0
	0	1	1	0	0	0	1
	0	1	1	0	0	0	1

Figure 9.01 Seven bytes to be transmitted

The parity bit is set for each of the bytes, as in Figure 9.02. The most significant bit is set to achieve even parity.

Parity bits	Seven-bit codes						
0	1	0	1	0	0	1	1
1	0	1	1	0	0	0	1
1	1	0	1	1	0	0	0
1	0	0	1	1	1	0	0
1	0	1	1	0	0	1	0
1	0	1	1	0	0	0	1
1	0	1	1	0	0	0	1

Figure 9.02 Bytes with the parity bit set

An additional byte is then created and each bit is set as a parity bit for the bits at that bit position. This includes counting the parity bits in the seven bytes containing data. This is illustrated in Figure 9.03.

Parity bits	Seven-bit codes							
0	1	0	1	0	0	1	1	
1	0	1	1	0	0	0	1	
1	1	0	1	1	0	0	0	
1	0	0	1	1	1	0	0	
1	0	1	1	0	0	1	0	
1	0	1	1	0	0	0	1	
1	0	1	1	0	0	0	1	
0	0	0	1	0	1	0	0	← **Parity byte**

Figure 9.03 Parity byte added

The program then transmits the eight bytes in sequence.

(Continued)